大模型通识课 · 实验作业六

SmolVLM-256M 精准定位的 RSI 递归自进化实验

—— 无需训练检测器，用「智能体自纠错 + 自举数据」教会小模型定位物体 ——

实验名称	SmolVLM-256M 精准定位的 RSI 递归自进化
对应教材	第 11 章《大模型智能体》（DeepSeek Harness / Hermes 自进化闭环）
模型规模	HuggingFaceTB/SmolVLM-256M-Instruct（2.56 亿参数）
运行方式	纯 CPU 可完成，无需 GPU、无需 Colab
建议时长	2~3 学时（课内完成必做内容约 25~35 分钟）
产出	rsi_memory.json（自纠错记忆）、sft_data.json（自举数据集）、实验报告

一、实验背景与目标

实验五已经用 QLoRA 教会 SmolVLM-256M「看图说话」。但一个只会描述画面的模型，并不能回答「那个红色圆形到底在哪儿」——定位（grounding）要求模型输出目标在图像中的位置，是具身智能、文档理解、GUI 智能体的共同基础能力。

本实验不复用实验五的 YOLO 蒸馏方案：训练一个检测器要 10~20 分钟，且引入 ultralytics 依赖与中文路径编码坑。本实验改用教材第 11 章的智能体范式——DeepSeek Harness 的「感知 → 工具校验 → 自纠错 → 记忆」闭环与 Hermes 的自进化思想，把「谁来判断答案对错」这件事从外部检测器换成解析式验证器，再让模型用自己的纠错结果继续训练，形成 RSI（Recursive Self-Improvement，递归自进化）闭环。

1.1 三个教学目标

目标	具体要求	对应能力
目标 1：会定位	把目标位置表达成 4×4 网格坐标（第几行第几列），并训练模型稳定输出该格式	空间定位的离散化表达
目标 2：会纠错	实现「高亮格回灌自查 + 验证器反馈重答」两路纠错，统计纠错触发率与成功率	智能体的反馈闭环（教材图 11-6）
目标 3：会自进化	让通过验证的纠错样本进入 SFT 数据池，再训练、再评测，观察轮次间的指标变化	RSI 递归自进化（教材 Hermes 自进化闭环）

表 1　实验目标与教材能力的对应关系

1.2 与 YOLO 蒸馏版的取舍

维度	YOLO 蒸馏版（扩展）	RSI 版（本实验）
监督信号来源	YOLOv8n 训练 15 轮，mAP50≈0.94 的框作为教师信号	GT 框直接换算成网格坐标 + 解析式验证器复核
时间开销	YOLO 训练 10~20 分钟 + 依赖 ultralytics	零训练成本，验证器单次调用 < 0.1 秒
部署门槛	需联网下载 yolov8n.pt，ultralytics 对中文路径不友好	纯 PIL + torch，无额外依赖
教学重心	「学模型的能力迁移」	「智能体如何把错误变成数据」——更贴近第 11 章主题
局限	教师信号有噪声，且把能力外包给了检测器	解析式验证器只在纯色合成图上成立，真实场景需换成学习式检测器

表 2　两种方案的对比（实验报告需说明你为什么选 RSI 版）

二、实验原理

2.1 定位任务的形式化：把像素空间压缩成 16 个类别

让 2.56 亿参数的模型直接回归 4 个浮点像素坐标，几乎不可能学会。更稳妥的做法是离散化：把 256×256 图像切成 4×4 共 16 个网格，模型只需回答「第 2 行第 3 列」——输出空间从 256 个像素值降为 16 个类别，样本效率大幅提高。

> 判定规则：目标连通块的中心落在预测网格内 → 定位正确；同时用 IoU(预测网格, GT 框)作为更细的连续指标，观察模型「差多少」。

2.2 RSI 递归自进化闭环

一轮 RSI 由四个动作组成，代码中对应 rsi_round()：

步骤	动作	产物
① 采样	模型在 15 道评测题上直接作答，得到原始预测 pred1	模型答案
② 自查	把 pred1 渲染成紫色高亮格回灌给模型，让它自己判断「框对不对」并改口	pred2_self（自发改写）
③ 校验	解析式验证器用颜色连通块分析判定高亮格是否真的定位到目标；不通过则把结论写入提示词重答	pred2_tool（工具纠错）
④ 自举	通过验证的答案连同图像、问题写入 SFT 数据池，模型在扩充后的数据上继续训练	sft_data.json 增长

表 3　RSI 一轮闭环的四个动作

关键点：第 ④ 步的数据来源是「模型自己产出、且被验证器确认正确」的答案——这就是递归自进化的本质：模型既是学习者，也是数据生产者，而验证器负责把关。

2.3 解析式验证器：零训练的「真值」来源

验证器只做一件事：用 PIL 逐像素扫描目标颜色，做 8 邻域连通块分析，得到每个物体的包围盒；再检查「某个连通块的中心是否落在模型预测的网格内」。这既不训练任何网络，也不依赖任何预训练权重，单次调用耗时不到 0.1 秒。

> 必须诚实说明的局限：这套验证器只在本实验的纯色几何图上成立。真实场景需要换成学习式检测器——但闭环结构（模型作答 → 工具校验 → 纠错 → 自举再训练）完全不变，这正是智能体范式的价值：把感知能力外包给工具，把学习留给模型。

三、环境与依赖

表 4　环境与依赖（全部 CPU 可完成）

项目	版本 / 要求	说明

Python	3.10+	Anaconda 或 venv 均可
PyTorch	2.2+（CPU 版即可）	pip install torch，纯 CPU 安装，无需 CUDA 组件
transformers	4.44+	需要支持 SmolVLM 的版本（本课程环境 4.57）
peft	0.11+	QLoRA 的 LoRA 层实现
bitsandbytes	可选	仅 4bit 量化训练需要；本实验默认 fp32，CPU 上不装也不影响
Pillow / NumPy	随环境自带	合成数据与解析式验证器只依赖 Pillow
模型权重	HuggingFaceTB/SmolVLM-256M-Instruct	约 500 MB，国内建议设 HF_ENDPOINT=https://hf-mirror.com
显存 / 显卡	不需要	fp32 推理约需 1.2 GB 内存

安装命令：

```
pip install torch transformers peft accelerate pillow
$env:HF_ENDPOINT="https://hf-mirror.com"   # Windows PowerShell；Linux 用 export
```

注意：如果系统预设了 HF_HUB_OFFLINE=1，请先取消（$env:HF_HUB_OFFLINE="0"），否则模型无法下载。

四、实验步骤

4.1 第 1 步：生成合成数据集

运行 --mode make_data，脚本用固定随机种子绘制 40 张训练图与 15 张评测图（256×256，白底，4 色 × 圆/方，1~4 个物体），并在绘制时同步记录每个物体的 GT 包围盒到 hw6_imgs/gt.json。评测集种子固定为 42，全班结果可直接对齐。

4.2 第 2 步：生成种子 SFT 数据

--mode make_sft 把 GT 框换算成网格坐标，写出 15 条「图像 + 问题 + 标准答案」样本。注意：这一步没有用到任何外部模型或人工标注——真值来自绘制时的几何信息，这就是「零训练成本」的来源。

4.3 第 3 步：基线推理

--mode infer 逐题输出模型的原始回答（含定位题的 IoU），作为「训练前」的基线。

4.4 第 4 步：QLoRA SFT

--mode sft --steps 10 训练 10 步（CPU 约 2~5 分钟），只对答案 token 计算损失，LoRA 挂在 q/k/v/o_proj 上，终端会打印可训练参数占比（应约 0.5%），这是报告表 5 必须抄下的数字。

4.5 第 5 步：三阶段评测

--mode eval 依次跑「基线 / SFT / SFT + 验证器纠错」三个阶段，输出分题型正确率、逐题答案与纠错前后的变化。

4.6 第 6 步：RSI 递归自进化

--mode rsi --rounds 2 --steps 10 跑两轮递归：每轮先采样与纠错、再把通过的样本并入数据池、然后再训练、最后重新评测。

表 5　命令与产出对照

命令	作用	产出文件
python hw6_rsi_localize.py --mode make_data	生成 55 张合成图 + GT 框	hw6_imgs/gt.json
python hw6_rsi_localize.py --mode make_sft	GT 框 → 网格坐标种子数据	sft_data.json
python hw6_rsi_localize.py --mode infer	基线逐题推理	终端输出
python hw6_rsi_localize.py --mode sft --steps 10	QLoRA 训练 10 步	sft_adapter/
python hw6_rsi_localize.py --mode eval	基线 / SFT / 纠错 三阶段评测	终端输出
python hw6_rsi_localize.py --mode rsi --rounds 2 --steps 10	2 轮递归自进化	rsi_memory.json

五、结果记录与评分

5.1 结果记录表（实验报告必须填写）

表 6　三阶段评测结果（照抄程序输出，不要手算）

阶段	loc（定位 8 题）	count（数数 4 题）	exist（存在 3 题）	总计（15 题）	定位平均 IoU
基线（未微调）					
SFT（种子数据 10 步）					
SFT + 验证器纠错					
RSI 第 1 轮					
RSI 第 2 轮					

表 7　RSI 轮次指标（每一轮都要填）

轮次	数据池规模	VLM 直接答对	自查改写次数	验证器纠错次数	纠错后答对	轮末评测正确率
初始（种子	15		—	—	—	

数据）						
第 1 轮						
第 2 轮						

表 8　评分标准（总分 100 分）

评分项	分值	评分要点
实验环境与复现	10	能在一台纯 CPU 机器上完整跑通全链路；记录依赖版本与耗时
定位任务实现	20	网格坐标编码正确；解析器能兼容「第 X 行第 Y 列」「rXcY」等写法
RSI 闭环实现	25	高亮格回灌自查、验证器纠错、自举数据入池、下一轮再训练四个环节齐全
结果记录与分析	25	表 6、表 7 填写完整；分析能回答「提升来自微调还是来自工具纠错」
思考题与讨论	15	见第六节，能结合自己跑出的数据讨论
报告规范	5	含运行日志、图表、结论与反思

六、思考题

1. 本实验用「解析式验证器」代替 YOLO 检测器。请说明这种做法在教学上的优点，以及它在真实场景中的根本局限。如果换到真实照片，验证器应该换成什么？这时闭环还成立吗？

2. 表 6 中 loc 一列的提升主要来自哪一步：种子 SFT、验证器纠错，还是 RSI 自举？请用你自己的数据给出证据（提示：对比「SFT」与「SFT + 纠错」两行，再对比「RSI 第 1 轮」与「第 2 轮」两行）。

3. 模型自查（把高亮格回灌给模型）与验证器纠错（把颜色分析结论写进提示词）的效果有何不同？哪一种更符合教材第 11 章对智能体「工具使用」的定义？

4. 若第 2 轮的提升小于第 1 轮，可能的原因有哪些？（提示：数据池里混入了什么？验证器会不会也犯错？）

5. 为什么把定位目标离散成 4×4 网格，而不是让模型直接输出像素坐标？如果改成 8×8 或 16×16，对数据量、输出空间和精度的要求分别有什么影响？

6. 本实验里「真值」来自绘制时的几何信息。若换成真实数据、没有 GT，RSI 闭环还能启动吗？你会如何设计一个「无 GT 的启动方案」？

七、常见问题（FAQ）

Q1：跑完 15 题基线推理要多久？

答：纯 CPU 下每题约 10~30 秒，15 题约 5~8 分钟；加上 SFT（10 步，约 2~5 分钟）与三阶段评测，全流程约 25~35 分钟。有 GPU 时全流程可压到 10 分钟内。

Q2：定位题模型输出「我不知道」「图片里有几个……」怎么办？

答：这是 2.56 亿参数基座模型的正常表现（格式未对齐）。用正则 r"第\s*([1-4])\s*行\s*第\s*([1-4])\s*列" 解析，解析失败记为未命中——这正是 SFT 要解决的第一件事：先学会格式，再谈准确率。

Q3：验证器会不会也出错？

答：会。两种典型情形：① 目标跨格时，中心落在哪一格取决于「格中心规则」；② 同色物体颜色非常接近阈值时可能被并成一个连通块。所以验证器消息里写的是「重叠多少」这类可核查的量，而不是直接给答案——这也是「工具给证据、模型做决策」的设计要点。

Q4：RSI 两轮之后准确率反而下降？

答：先检查数据池里是否混入了「答案格式对但内容错」的样本（验证器只判位置，不判措辞），再检查学习率是否过大导致过拟合。建议每轮后都把评测表贴出来对比，不要只看最后一轮。

Q5：Windows 报 UnicodeEncodeError 或模型下载失败？

答：前者是控制台编码问题（chcp 65001 或把日志重定向到文件），后者通常是 HF_HUB_OFFLINE=1 被预设或网络问题，用 HF_ENDPOINT=https://hf-mirror.com 并取消离线变量即可。

Q6：可以不用 CPU 吗？

答：可以，脚本会自动使用 CUDA（torch.cuda.is_available()）。但本实验所有指标在 CPU 上即可获得，不建议为追求速度引入 GPU 环境。

Q7：sft_data.json 会被覆盖吗？

答：make_sft 会覆盖它；RSI 每轮结束时也会重写它（把本轮新通过验证的样本并进去）。想保留中间版本请自行备份。

Q8：为什么定位题只有 8 道？

答：题量受 CPU 推理时间限制。定位题每题要跑 2~3 次生成（作答 + 自查 + 纠错），8 道已能看出趋势；有余力的同学可把 EVAL_N 改成 20，并相应增大 CPU 时间预算。

八、拓展任务（选做其一）

A. 网格精细化：把 GRID 改为 8，观察定位准确率与 IoU 的变化，并讨论「离散粒度」与「小模型容量」的权衡。

B. 自进化实验（教材 Hermes 闭环）：把通过验证的纠错样本再训一轮，观察模型在自举数据上是否真的变好，并分析「自举数据分布过窄」带来的退化风险。

C. 换验证器：把解析式验证器换成 YOLOv8n 预训练模型（不训练，只做检测），对比两种验证器在纠错成功率上的差异，讨论「工具精度如何决定智能体上限」。

九、实验报告要求

1. 报告正文不超过 3 页，须含：环境与耗时、命令清单、完整运行日志（终端输出截图或日志节选）、表 6 与表 7 的填写结果、一张自绘图（建议画三轮的定位正确率折线或数据池规模增长曲线）、结论与反思。

2. 结论部分必须回答：「本实验中定位能力的提升，究竟来自参数更新（微调）还是来自推理时的工具纠错？两者的作用边界在哪里？」请用表 6、表 7 的数据支撑，不要空谈。

3. 必须如实记录失败：若某轮指标下降、验证器误判、模型输出异常，均需在报告「问题与调试」一节写明现象、原因与处理方式。

附录　参考实现（hw6_rsi_localize.py）

以下为完整参考代码，可直接运行；也可只阅读关键函数：cell_of（网格换算）、color_components（连通块分析）、verify_cell（验证器）、overlay（高亮回灌）、rsi_round（RSI 一轮）、rsi_loop（递归主循环）。

```
# -*- coding: utf-8 -*-
"""
SmolVLM-256M 精准定位的 RSI 递归自进化实验 —— 实验作业六参考框架（改良版，纯 CPU）

设计要点（相比 YOLO 蒸馏版）：
  ① 不再训练任何检测器：定位的「真值」由合成数据的 GT 框直接给出，零训练开销；
  ② 验证器是解析式的颜色掩码分析（PIL 像素统计，毫秒级），不是学习模型；
  ③ 智能体侧用 RSI（Recursive Self-Improvement，递归自进化）：
     预测 → 渲染高亮格回灌 → 自查/验证器纠错 → 通过验证的样本进入数据池 → 再训练 → 下一轮
  ④ 输出空间用 4×4 网格（16 类）而非像素坐标，便于 256M 小模型学习。

常用命令：
python hw6_rsi_localize.py --mode make_data   # 生成合成数据集（首次运行自动执行）
python hw6_rsi_localize.py --mode make_sft    # 由 GT 生成种子 SFT 数据（定位用网格坐标）
python hw6_rsi_localize.py --mode infer       # 基线推理 15 题
python hw6_rsi_localize.py --mode sft --steps 10   # QLoRA SFT（产出 ./sft_adapter）
python hw6_rsi_localize.py --mode eval        # 三阶段：基线 / SFT / SFT+纠错
python hw6_rsi_localize.py --mode rsi --rounds 2     # RSI 递归自进化多轮
"""
import argparse, json, os, random, re
import torch
from PIL import Image, ImageDraw

MODEL_ID = "HuggingFaceTB/SmolVLM-256M-Instruct"
IMG_DIR, SFT_DIR, MEM_JSON = "hw6_imgs", "sft_adapter", "rsi_memory.json"
GRID, IMG = 4, 256
CELL = IMG // GRID
COLORS = {"红": (220, 50, 50), "蓝": (50, 80, 220), "绿": (40, 170, 70), "黄": (230, 190, 40)}
TRAIN_N, EVAL_N = 40, 15

# ============================================================
# 1. 合成数据集：与实验五同风格的「颜色-形状」几何图形，绘制时记录 GT 框
# ============================================================
def draw_scene(rnd):
    items = []
    for _ in range(rnd.randint(1, 4)):
        color = rnd.choice(list(COLORS))
        shape = rnd.choice(["圆", "方"])
        s = rnd.randint(34, 54)
        x, y = rnd.randint(8, IMG - s - 8), rnd.randint(8, IMG - s - 8)
        items.append((f"{color}{shape}", [x, y, x + s, y + s]))
    return items

def render(items, path):
    im = Image.new("RGB", (IMG, IMG), "white")
    d = ImageDraw.Draw(im)
    for cls, box in items:
        color, shape = cls[0], cls[1:]
        if shape == "圆":
            d.ellipse(box, outline=COLORS[color], width=4)
        else:
```

```
            d.rectangle(box, outline=COLORS[color], width=4)
    im.save(path)

def make_dataset():
    os.makedirs(f"{IMG_DIR}/train", exist_ok=True)
    os.makedirs(f"{IMG_DIR}/eval", exist_ok=True)
    gt = {}
    rnd = random.Random(2026)
    for i in range(TRAIN_N):
        items = draw_scene(rnd)
        render(items, f"{IMG_DIR}/train/t{i:02d}.png")
        gt[f"t{i:02d}"] = items
    rnd = random.Random(42)                          # 评测集种子固定，人人可复现
    for i in range(EVAL_N):
        items = draw_scene(rnd)
        render(items, f"{IMG_DIR}/eval/e{i:02d}.png")
        gt[f"e{i:02d}"] = items
    json.dump(gt, open(f"{IMG_DIR}/gt.json", "w", encoding="utf-8"),
              ensure_ascii=False, indent=1)
    print(f"[data] 已生成 {TRAIN_N} 张训练图 + {EVAL_N} 张评测图 → {IMG_DIR}/")

def load_gt():
    if not os.path.exists(f"{IMG_DIR}/gt.json"):
        make_dataset()
    return json.load(open(f"{IMG_DIR}/gt.json", encoding="utf-8"))

# ============================================================
# 2. 定位编码：4×4 网格（行-列），把 256×256 像素压缩成 16 个类别
# ============================================================
def cell_of(box):
    """GT 框 → 中心所在网格 (row, col)"""
    cx, cy = (box[0] + box[2]) / 2, (box[1] + box[3]) / 2
    return min(GRID - 1, int(cy // CELL)), min(GRID - 1, int(cx // CELL))

def cell_box(r, c):
    return [c * CELL, r * CELL, (c + 1) * CELL, (r + 1) * CELL]

def cell_text(r, c):
    return f"第{r+1}行第{c+1}列"

def parse_cell(text):
    """从模型输出里解析 (row, col)，支持「第 2 行第 3 列」「r2c3」「2,3」等写法"""
    t = text.strip()
    m = re.search(r"第\s*([1-4])\s*行\s*第\s*([1-4])\s*列", t)
    if m:
        return int(m.group(1)) - 1, int(m.group(2)) - 1
    m = re.search(r"r\s*([1-4])\s*c\s*([1-4])", t, re.I)
    if m:
        return int(m.group(1)) - 1, int(m.group(2)) - 1
    m = re.search(r"([1-4])\s*[,\-，、]\s*([1-4])", t)
    if m:
        return int(m.group(1)) - 1, int(m.group(2)) - 1
    return None

def iou(a, b):
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
```

```
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    ua = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / ua if ua > 0 else 0.0

# ============================================================
# 3. 解析式验证器：颜色掩码分析（零训练，毫秒级）
#    回答两件事：目标颜色的像素是否落在指定网格内（验证定位）、共有几个连通块（数数）
# ============================================================
def color_components(img, color):
    """解析式感知：提取某颜色的连通块（每个连通块 = 一个物体），返回 [(bbox, area), ...]"""
    im = img.convert("RGB")
    px = im.load()
    ref = COLORS[color]
    W, H = im.size
    def near(p):
        return sum((p[i] - ref[i]) ** 2 for i in range(3)) < 9000
    s = {(x, y) for y in range(H) for x in range(W) if near(px[x, y])}
    comps = []
    while s:
        seed = s.pop()
        stack, pts = [seed], [seed]
        while stack:
            x, y = stack.pop()
            for dx in (-1, 0, 1):
                for dy in (-1, 0, 1):
                    q = (x + dx, y + dy)
                    if q in s:
                        s.discard(q); stack.append(q); pts.append(q)
        xs = [p[0] for p in pts]; ys = [p[1] for p in pts]
        comps.append(([min(xs), min(ys), max(xs) + 1, max(ys) + 1], len(pts)))
    return comps

def color_mask_stats(img, color, cell=None):
    """统计某颜色：全图连通块数、像素总数；若给定网格，另算该格内像素占比"""
    im = img.convert("RGB")
    px = im.load()
    ref = COLORS[color]
    W, H = im.size
    def near(p):
        return sum((p[i] - ref[i]) ** 2 for i in range(3)) < 9000
    pts = [(x, y) for y in range(H) for x in range(W) if near(px[x, y])]
    total = len(pts)
    inside = 0
    if cell is not None:
        cb = cell_box(*cell)
        inside = sum(1 for (x, y) in pts if cb[0] <= x < cb[2] and cb[1] <= y < cb[3])
    return {"total": total, "inside": inside, "components": len(color_components(img, color))}

# ============================================================
# 4. 问题集：loc=网格定位（主攻）/ count=数数 / exist=存在性
# ============================================================
def name_of(cls):
    return f"{cls[0]}色{cls[1:]}形"

def build_questions(gt):
    qs = []
    for i in range(EVAL_N):
```

```
        eid = f"e{i:02d}"
        items = gt[eid]
        present = sorted({c for c, _ in items})
        cls = present[i % len(present)]
        box = next(b for c, b in items if c == cls)
        r, c = cell_of(box)
        qs.append(dict(id=f"L{i+1:02d}", type="loc", img=eid, target=cls,
                       q=f"图中的{name_of(cls)}位于第几行第几列？请用「第 X 行第 Y 列」回答（共{GRID}行{GRID}
列）。",
                       gold=cell_text(r, c), cell=[r, c], box=box))
        same = sum(1 for cc, _ in items if cc == cls)
        qs.append(dict(id=f"C{i+1:02d}", type="count", img=eid, target=cls,
                       q=f"图中有几个{name_of(cls)}？", gold=f"{same}个", n=same))
        qs.append(dict(id=f"E{i+1:02d}", type="exist", img=eid, target=cls,
                       q=f"图里有没有{name_of(cls)}？", gold="有"))
    # 取 8 道定位 + 4 道数数 + 3 道存在性 = 15 题
    locs = [q for q in qs if q["type"] == "loc"][:8]
    cnts = [q for q in qs if q["type"] == "count"][:4]
    exs = [q for q in qs if q["type"] == "exist"][:3]
    return locs + cnts + exs

def judge(q, pred):
    """宽松判分：定位看网格是否命中（并记录 IoU），数数看数字，存在性看有无"""
    p = pred.strip().lower()
    if q["type"] == "loc":
        c = parse_cell(p)
        if c is None:
            return False
        return c == tuple(q["cell"])
    if q["type"] == "count":
        nums = re.findall(r"\d+", p)
        return bool(nums) and int(nums[0]) == q["n"]
    yes = ("有" in p and "没有" not in p.replace("有没有", "")) or "yes" in p
    no = ("没有" in p.replace("有没有", "")) or "no" in p or "not" in p
    return yes if q["gold"] == "有" else no

def loc_iou(q, pred):
    c = parse_cell(pred)
    return iou(cell_box(*c), q["box"]) if c else 0.0

# ============================================================
# 5. 模型封装：加载 / 提示 / 生成 / QLoRA SFT
# ============================================================
def load_model(attach_lora=False):
    from transformers import AutoModelForVision2Seq, AutoProcessor
    processor = AutoProcessor.from_pretrained(MODEL_ID)
    model = AutoModelForVision2Seq.from_pretrained(MODEL_ID, torch_dtype=torch.float32)
    if attach_lora:
        from peft import LoraConfig, get_peft_model
        lora = LoraConfig(r=8, lora_alpha=16, lora_dropout=0.05,
                          target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
                          task_type="CAUSAL_LM")
        model = get_peft_model(model, lora)
        model.print_trainable_parameters()          # ← 报告要抄的参数占比
    model.to("cuda" if torch.cuda.is_available() else "cpu")
    return processor, model
```

```
def build_prompt(processor, q):
    msgs = [{"role": "user", "content": [{"type": "image"}, {"type": "text", "text": q}]}]
    return processor.apply_chat_template(msgs, add_generation_prompt=True)

@torch.no_grad()
def answer(processor, model, img, q, hint=None, max_new=24):
    """hint 不为空时把验证器反馈拼进问题——Harness/RSI 纠错的关键动作"""
    text = q if hint is None else f"{q}\n（校验提示：{hint}）"
    inputs = processor(text=[build_prompt(processor, text)], images=[img],
                       return_tensors="pt").to(model.device)
    out = model.generate(**inputs, max_new_tokens=max_new, do_sample=False)
    return processor.decode(out[0][inputs["input_ids"].shape[1]:],
                            skip_special_tokens=True).strip()

def make_sft(gt):
    """种子 SFT 数据：定位题答案直接由 GT 框换算成网格坐标（零人工标注、零外部模型）"""
    data = []
    for q in build_questions(gt):
        if q["type"] == "loc":
            ans = q["gold"]
        elif q["type"] == "count":
            ans = q["gold"]
        else:
            ans = "有"
        data.append({"img": q["img"], "prompt": q["q"], "answer": ans})
    json.dump(data, open("sft_data.json", "w", encoding="utf-8"),
              ensure_ascii=False, indent=1)
    print(f"[sft] 种子数据 {len(data)} 条（定位答案由 GT 框换算）→ sft_data.json")
    return data

def sft_step(model, processor, opt, steps=10, log_every=5, data=None):
    data = data or json.load(open("sft_data.json", encoding="utf-8"))
    dev = next(model.parameters()).device
    model.train()
    for i in range(steps):
        s = data[i % len(data)]
        img = Image.open(f"{IMG_DIR}/eval/{s['img']}.png").convert("RGB")
        full = processor(text=[build_prompt(processor, s["prompt"]) + s["answer"]],
                         images=[img], return_tensors="pt").to(dev)
        with torch.no_grad():
            plen = processor(text=[build_prompt(processor, s["prompt"])],
                             images=[img], return_tensors="pt")["input_ids"].shape[1]
        labels = full["input_ids"].clone()
        labels[:, :plen] = -100                    # 只学答案 token
        full["labels"] = labels
        loss = model(**full).loss
        loss.backward(); opt.step(); opt.zero_grad()
        if (i + 1) % log_every == 0:
            print(f"[sft] step{i+1}/{steps} loss={loss.item():.4f}")
    model.save_pretrained(SFT_DIR)
    print(f"[sft] 适配器已保存到 ./{SFT_DIR}（本轮训练样本 {len(data)} 条）")

# ============================================================
# 6. RSI 递归自进化闭环
#    预测 → 渲染高亮格回灌自查 → 验证器纠错 → 通过验证的样本入池 → 再训练 → 下一轮
# ============================================================
def overlay(img, cell, path):
```

```
    """把模型预测的网格渲染成高亮方框，作为「可视化工具输出」回灌给模型"""
    im = img.copy()
    d = ImageDraw.Draw(im)
    cb = cell_box(*cell)
    d.rectangle(cb, outline=(255, 0, 255), width=5)
    d.rectangle([cb[0]+6, cb[1]+6, cb[2]-6, cb[3]-6], outline=(255, 0, 255), width=2)
    im.save(path)
    return path

def verify_cell(img, q, cell):
    """解析式验证器：判断预测网格是否真的定位到了目标物体
    判据：目标连通块的中心点落在该网格内（跨格物体也成立），或网格覆盖物体 ≥50% 面积"""
    comps = color_components(img, q["target"][0])
    if not comps:
        return False, "该颜色在图中不存在"
    cb = cell_box(*cell)
    best_overlap = 0.0
    for box, _ in comps:
        cx, cy = (box[0] + box[2]) / 2, (box[1] + box[3]) / 2
        if cb[0] <= cx < cb[2] and cb[1] <= cy < cb[3]:
            return True, "高亮格确实定位到了目标（目标中心在该格内）"
        x1, y1 = max(cb[0], box[0]), max(cb[1], box[1])
        x2, y2 = min(cb[2], box[2]), min(cb[3], box[3])
        inter = max(0, x2 - x1) * max(0, y2 - y1)
        area = (box[2] - box[0]) * (box[3] - box[1])
        best_overlap = max(best_overlap, inter / area if area else 0.0)
    if best_overlap >= 0.5:
        return True, f"高亮格覆盖了目标 {best_overlap:.0%} 的面积"
    return False, f"高亮格没有定位到目标（重叠仅 {best_overlap:.0%}），请重新给出位置"

def rsi_round(processor, model, gt, QUESTIONS, round_idx, mem):
    """一轮 RSI：模型作答 → 渲染回灌自查 → 验证器判定 → 收集可用样本"""
    print(f"\n===== RSI 第 {round_idx} 轮 =====")
    new_data, n_ok, n_self, n_tool, ious = [], 0, 0, 0, []
    for q in QUESTIONS:
        img = Image.open(f"{IMG_DIR}/eval/{q['img']}.png").convert("RGB")
        pred1 = answer(processor, model, img, q["q"])
        ok1 = judge(q, pred1)
        if q["type"] == "loc":
            ious.append(loc_iou(q, pred1))
        n_ok += ok1
        rec = {"id": q["id"], "type": q["type"], "pred1": pred1, "ok1": ok1}
        if q["type"] == "loc" and not ok1:
            # ① 智能体自查：把高亮格渲染回图像，让模型自己检查
            c = parse_cell(pred1)
            if c:
                ov = overlay(img, c, f"{IMG_DIR}/ov_{q['id']}.png")
                ov_img = Image.open(ov).convert("RGB")
                pred2 = answer(processor, model, ov_img,
                               f"图中紫色高亮框是上一次给出的位置。{q['q']}\n"
                               "若高亮框没有圈住目标，请直接给出正确位置。", max_new=16)
                if parse_cell(pred2) != c:
                    n_self += 1
                    rec["pred2_self"] = pred2
            # ② 验证器纠错：解析式颜色掩码判定高亮格是否成立
            c = parse_cell(pred1) or (0, 0)
            passed, msg = verify_cell(img, q, c)
```

```
            if not passed:
                n_tool += 1
                pred3 = answer(processor, model, img, q["q"], hint=msg)
                rec["hint"] = msg
                rec["pred2_tool"] = pred3
                if judge(q, pred3):
                    new_data.append({"img": q["img"], "prompt": q["q"], "answer": pred3})
        mem.append({**rec, "round": round_idx})
    print(f"[rsi] 轮 {round_idx}：VLM 答对 {n_ok}/{len(QUESTIONS)}，"
          f"自查改写 {n_self} 次，验证器纠错 {n_tool} 次，"
          f"新增可用样本 {len(new_data)} 条")
    if ious:
        print(f"[rsi] 轮 {round_idx}：定位平均 IoU = {sum(ious)/len(ious):.3f}")
    return new_data

def rsi_loop(rounds=2, steps=10):
    """完整的 RSI 递归：每轮 = 纠错采样 → 数据池扩充 → 再训练 → 评测"""
    gt = load_gt()
    QUESTIONS = build_questions(gt)
    processor, model = load_model(attach_lora=True)
    from torch.optim import AdamW
    opt = AdamW([p for p in model.parameters() if p.requires_grad], lr=1e-4)
    pool = make_sft(gt)
    mem = []
    print(f"[rsi] 初始数据池 {len(pool)} 条，开始 {rounds} 轮递归自进化")
    for r in range(1, rounds + 1):
        new_data = rsi_round(processor, model, gt, QUESTIONS, r, mem)
        if new_data:
            pool = pool + new_data
            json.dump(pool, open("sft_data.json", "w", encoding="utf-8"),
                      ensure_ascii=False, indent=1)
        sft_step(model, processor, opt, steps=steps, data=pool)
        evaluate(processor, model, f"RSI 轮{r}")
    json.dump(mem, open(MEM_JSON, "w", encoding="utf-8"), ensure_ascii=False, indent=1)
    print(f"[rsi] 记忆写入 {MEM_JSON}（{len(mem)} 条）；数据池最终 {len(pool)} 条")

# ============================================================
# 7. 三阶段评测：基线 / SFT / SFT+验证器纠错
# ============================================================
def evaluate(processor, model, tag):
    gt = load_gt()
    QUESTIONS = build_questions(gt)
    print(f"\n===== {tag} × {len(QUESTIONS)} 题 =====")
    ok_by_type = {}
    for tp in ("loc", "count", "exist"):
        qs = [q for q in QUESTIONS if q["type"] == tp]
        ok = 0
        for q in qs:
            img = Image.open(f"{IMG_DIR}/eval/{q['img']}.png").convert("RGB")
            pred = answer(processor, model, img, q["q"])
            hit = judge(q, pred)
            ok += hit
            if tp == "loc":
                print(f"[{q['id']}] {'√' if hit else '×'} A: {pred[:40]}  gold: {q['gold']}
IoU={loc_iou(q, pred):.2f}")
        ok_by_type[tp] = (ok, len(qs))
        print(f"{tag:<14}{tp:<7}正确 {ok}/{len(qs)}")
```

```
    total = sum(o for o, _ in ok_by_type.values())
    n = sum(l for _, l in ok_by_type.values())
    print(f"{tag:<14}总计     正确 {total}/{n}\n")
    return total

def eval_with_correction(processor, model, tag):
    """SFT + 验证器纠错：错题把验证器反馈注入提示词重答一次"""
    gt = load_gt()
    QUESTIONS = build_questions(gt)
    print(f"\n===== {tag} + 验证器纠错 × {len(QUESTIONS)} 题 =====")
    ok1 = ok2 = 0
    for q in QUESTIONS:
        img = Image.open(f"{IMG_DIR}/eval/{q['img']}.png").convert("RGB")
        p1 = answer(processor, model, img, q["q"])
        hit1 = judge(q, p1)
        ok1 += hit1
        if hit1:
            p2, hit2 = p1, True
        else:
            if q["type"] == "loc":
                c = parse_cell(p1) or (0, 0)
                _, msg = verify_cell(img, q, c)
            else:
                st = color_mask_stats(img, q["target"][0])
                msg = (f"图中该颜色共 {st['components']} 处" if q["type"] == "count"
                       else f"图中该颜色像素数 {st['total']}")
            p2 = answer(processor, model, img, q["q"], hint=msg)
            hit2 = judge(q, p2)
        ok2 += hit2
        mark = "√" if hit2 else "×"
        print(f"[{q['id']}·{q['type']}] {mark} A1: {p1[:30]} → A2: {p2[:30]}  gold: {q['gold']}")
    print(f"{tag:<14}纯 VLM 正确 {ok1}/{len(QUESTIONS)}   纠错后正确 {ok2}/{len(QUESTIONS)}\n")
    return ok1, ok2

# ============================================================
# 8. 主入口
# ============================================================
def main():
    ap = argparse.ArgumentParser(description="实验作业六：SmolVLM-256M 精准定位的 RSI 递归自进化")
    ap.add_argument("--mode", choices=["make_data", "make_sft", "infer", "sft", "eval", "rsi"],
                    default="infer")
    ap.add_argument("--steps", type=int, default=10, help="每轮 SFT 步数（CPU 建议 5~10）")
    ap.add_argument("--rounds", type=int, default=2, help="RSI 递归轮数")
    args = ap.parse_args()
    gt = load_gt()
    QUESTIONS = build_questions(gt)
    if args.mode == "make_data":
        return
    if args.mode == "make_sft":
        make_sft(gt); return
    if args.mode == "infer":
        processor, model = load_model()
        for q in QUESTIONS:
            img = Image.open(f"{IMG_DIR}/eval/{q['img']}.png").convert("RGB")
            pred = answer(processor, model, img, q["q"])
            mark = "√" if judge(q, pred) else "×"
            extra = f"  IoU={loc_iou(q, pred):.2f}" if q["type"] == "loc" else ""
```

```
            print(f"[{q['id']}·{q['type']}] {mark} Q: {q['q'][:30]}  A: {pred[:50]}（gold:
{q['gold']}）{extra}")
        return
    if args.mode == "sft":
        if not os.path.exists("sft_data.json"):
            make_sft(gt)
        processor, model = load_model(attach_lora=True)
        from torch.optim import AdamW
        opt = AdamW([p for p in model.parameters() if p.requires_grad], lr=1e-4)
        sft_step(model, processor, opt, steps=args.steps)
        return
    if args.mode == "eval":
        processor, model = load_model()
        evaluate(processor, model, "基线")
        if os.path.exists(SFT_DIR):
            from peft import PeftModel
            model = PeftModel.from_pretrained(model, SFT_DIR).to(model.device)
            evaluate(processor, model, "SFT")
            eval_with_correction(processor, model, "SFT")
        print("提示：把三阶段结果与纠错统计抄进报告表 3。")
        return
    if args.mode == "rsi":
        rsi_loop(rounds=args.rounds, steps=args.steps); return

if __name__ == "__main__":
    main()
```